



ZAAWANSOWANE PROGRAMOWANIE W JĘZYKU C++

RAPORT Z WYKONANIA PROJEKTU

PLANE MANAGER

Hubert Lech — 177827 — L09

2026-05-17

Spis treści

1	Wstęp	2
2	Baza danych	3
2.1	Samoloty (planes)	4
2.1.1	Create	5
2.1.2	Read	5
2.1.3	Update	6
2.1.4	Delete	7
2.2	Lotniska (airports)	7
2.2.1	Create	8
2.2.2	Read	8
2.2.3	Update	9
2.2.4	Delete	9
2.3	Loty (flights)	9
2.3.1	Create	10
2.3.2	Read	10
2.3.3	Update	12
2.3.4	Delete	13
3	Struktura aplikacji	14
4	Działanie aplikacji	17
4.1	Ekran główny: nawigacja pomiędzy modułami	17
4.2	Samoloty: lista floty (odczyt danych)	18
4.3	Samoloty: dodawanie nowej maszyny (Create)	18
4.4	Samoloty: edycja i usuwanie (Update/Delete)	19
4.5	Samoloty: szczegóły i zarządzanie zdjęciem	20
4.6	Lotniska: lista i zarządzanie rekordami (Read/Update/Delete)	20
4.7	Lotniska: dodawanie z pomocą linku Google Maps i odpowiedzi	21
4.8	Loty: harmonogram (Read) i anulowanie rezerwacji (Delete)	22
4.9	Loty: planowanie i edycja lotu (Create/Update) oraz wybór daty	23
4.10	Mapa lotów	24
4.11	Statystyka floty	25
5	Podsumowanie	27

1 Wstęp

Plane Manager to desktopowa aplikacja do zarządzania niewielką flotą lotniczą, zaprojektowana tak, aby w prosty i czytelny sposób zarządzać informacjami o samolotach oraz ich wykorzystaniu w zaplanowanych lotach. Z perspektywy użytkownika aplikacja działa jak mały panel administracyjny: można dodać samolot, uzupełnić jego parametry techniczne (np. długość, liczba silników, pojemność pasażerska, prędkość i pułap), zmienić status (np. dostępny lub w serwisie), a następnie zaplanować lot pomiędzy wybranymi lotniskami. Podczas rezerwacji i edycji lotów aplikacja pilnuje porządku w terminarzu: sprawdza, czy daty są poprawne oraz czy nie dochodzi do nakładania się rezerwacji dla tego samego samolotu; dodatkowo uwzględnia sytuację, w której samolot jest wyłączony z użytku (np. znajduje się w serwisie).

Poza samym planowaniem lotów dostępny jest też widok podsumowań: statystyki floty (ile maszyn jest dostępnych, w locie lub w serwisie), a także informacje takie jak najdłuższy i najkrótszy samolot czy największa i najmniejsza liczba pasażerów. Dzięki temu jednym spojrzeniem można ocenić, w jakiej kondycji jest flota i jakie maszyny wyróżniają się parametrami.

Projekt został zrealizowany w oparciu o framework Qt. Logika aplikacji i komunikacja z bazą danych zostały napisane w C++, natomiast interfejs użytkownika przygotowano w QML, co pozwoliło uzyskać responsywny i miły dla oka układ widoków. Dane przechowywane są w relacyjnej bazie PostgreSQL uruchomionej w środowisku Supabase. Dzięki temu rozwiązaniu aplikacja jest gotowa do dalszej rozbudowy (np. o kolejne encje, takie jak załoga czy trasy) i może być łatwo przenoszona pomiędzy komputerami bez konieczności ręcznej konfiguracji bazy danych (parametry dostępu są pobierane ze zmiennych środowiskowych).

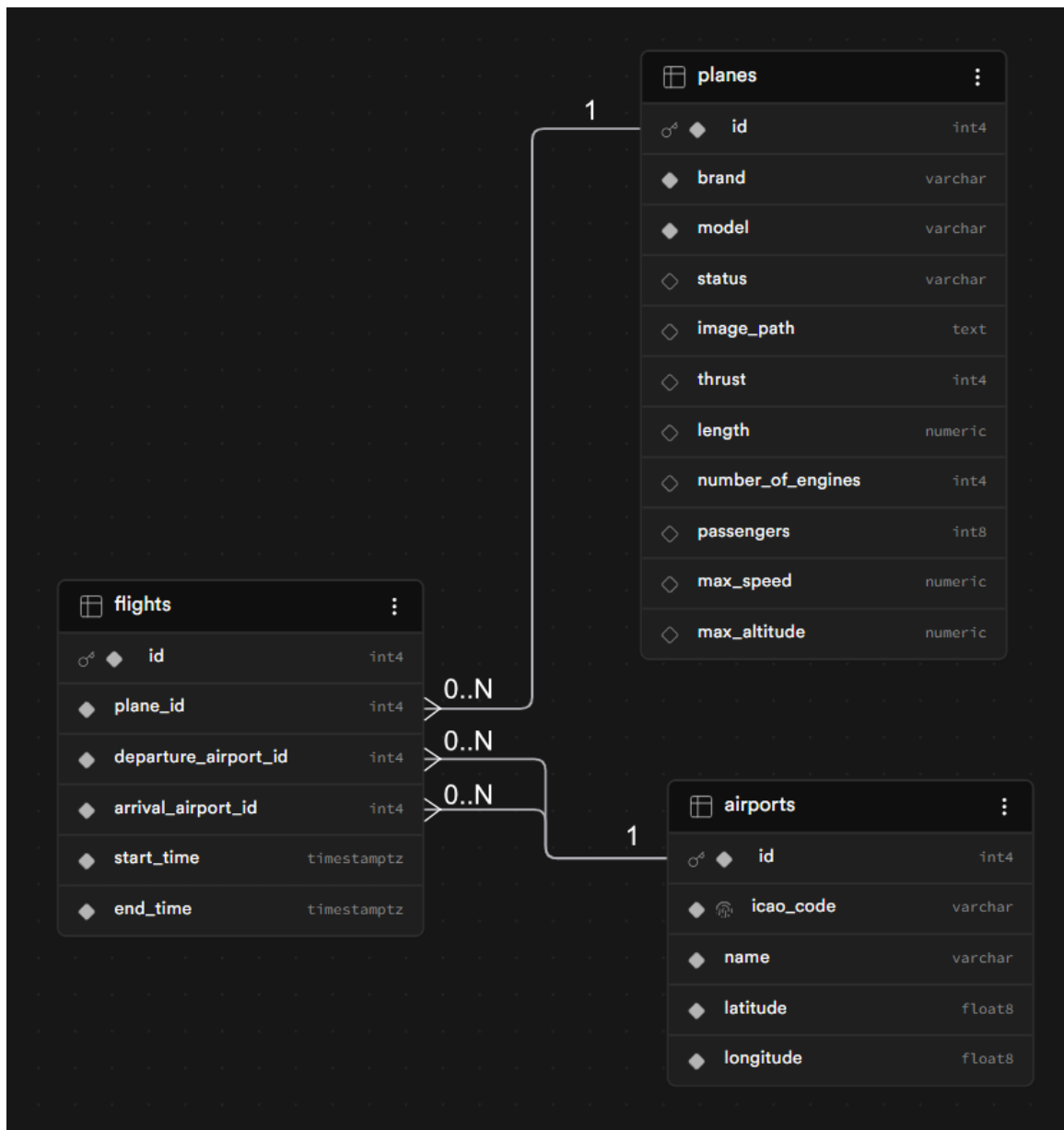
2 Baza danych

Po uruchomieniu aplikacji nawiązywane jest połączenie z bazą danych PostgreSQL działającą w Supabase. Za ten etap odpowiada klasa `DatabaseManager`. Parametry dostępu nie są wpisane na stałe w kodzie, aplikacja pobiera je ze zmiennych środowiskowych `.ENV`, co ułatwia przenoszenie projektu pomiędzy komputerami i ogranicza ryzyko przypadkowego ujawnienia danych dostępowych w repozytorium.

W projekcie baza danych składa się z trzech głównych tabel, które są niezbędne w zarządzaniu flotą lotniczą i planowaniem lotów:

- **planes** — przechowuje informacje o samolotach (marka, model, status oraz parametry techniczne takie jak długość, liczba silników, pojemność, prędkość maksymalna i pułap). Rekordy identyfikowane są kluczem głównym `id`.
- **airports** — zawiera lotniska wraz z kodem ICAO, nazwą oraz współrzędnymi geograficznymi (szerokość i długość). Rekordy identyfikowane są kluczem `id`.
- **flights** — reprezentuje zaplanowane loty: wskazuje samolot (`plane_id`) oraz lotniska wylotu i przylotu (`departure_airport_id`, `arrival_airport_id`), a także przedział czasowy lotu (`start_time`, `end_time`).

Schemat bazy przedstawiono na diagramie 1. W czasie implementacji ważne było również poprawne przetwarzanie czasu — wartości czasowe zapisywane są w bazie w UTC (aplikacja dokonuje konwersji na UTC przy zapisie oraz na czas lokalny przy prezentacji), a przy dodawaniu/edycji lotu wykonywana jest walidacja terminów, w tym wykrywanie nakładania się rezerwacji dla tego samego samolotu.



Rysunek 1: Diagram relacji bazy danych (ERD)

Poniżej przedstawiono przykładowe fragmenty kodu odpowiedzialne za operacje CRUD (Create, Read, Update, Delete) na głównych encjach aplikacji: samolotach, lotniskach oraz lotach.

2.1 Samoloty (*planes*)

Samoloty są jedną z kluczowych encji w aplikacji, dlatego operacje CRUD na tabeli **planes** są bardziej rozbudowane. Oprócz podstawowych pól (marka, model, status) tabela zawiera również parametry techniczne (długość, liczba silników, pojemność, prędkość maksymalna i pułap), które są istotne dla planowania lotów. Dodatkowo podczas odczytu listy samolotów uwzględniany jest aktualny status (dostępny/w locie/w serwisie) na podstawie danych z tabeli **flights**.

2.1.1 Create

Dodawanie samolotu realizuje metoda `addPlane` (listing 1). Zapytanie `INSERT` jest parametryzowane, a wartości pól są wiązane przez `bindValue`. Dzięki temu interfejs QML może przekazać komplet parametrów technicznych bez ręcznego składania SQL, co zapobiega również atakom typu SQL Injection.

```

1  bool PlaneManager::addPlane(const QString &brand, const QString
    &model, const QString &status,
2                                int thrust, double length, int
                                numberOfEngines,
3                                int passengers, double maxSpeed, double
                                maxAltitude) {
4
5      QSqlQuery query;
6      query.prepare("INSERT INTO planes (brand, model, status, thrust,
        length, number_of_engines, passengers, max_speed, max_altitude,
        image_path) "
7                      "VALUES (:brand, :model, :status, :thrust, :length,
        :numberOfEngines, :passengers, :maxSpeed,
        :maxAltitude, '')");
8      query.bindValue(":brand", brand);
9      query.bindValue(":model", model);
10     query.bindValue(":status", status);
11     query.bindValue(":thrust", thrust);
12     query.bindValue(":length", length);
13     query.bindValue(":numberOfEngines", numberOfEngines);
14     query.bindValue(":passengers", passengers);
15     query.bindValue(":maxSpeed", maxSpeed);
16     query.bindValue(":maxAltitude", maxAltitude);
17
18     if (!query.exec()) {
19         qDebug() << "Bład SQL przy dodawaniu samolotu:" <<
            query.lastError().text();
20         return false;
21     }
22
23     qDebug() << "Pomyślnie dodano samolot:" << brand << model;
24     return true;
25 }

```

Listing 1: Create dla samolotów: `addPlane`

2.1.2 Read

Odczyt listy samolotów odbywa się w `getAllPlanes` (listing 2) i zwraca `QVariantList` map, co jest bezpośrednio kompatybilne z QML.

```

1  QVariantList PlaneManager::getAllPlanes() {
2      QVariantList list;
3      QSqlQuery query(
4          "SELECT p.id, p.brand, p.model, p.thrust, p.length,
        p.number_of_engines, p.passengers, p.max_speed,
        p.max_altitude, "

```

```

5      "COALESCE(p.image_path, '') as image_path, "
6      "CASE "
7      "      WHEN EXISTS ("
8      "          SELECT 1 FROM flights f "
9      "          WHERE f.plane_id = p.id "
10     "          AND f.start_time <= CURRENT_TIMESTAMP "
11     "          AND f.end_time >= CURRENT_TIMESTAMP"
12     "      ) THEN 'W locie' "
13     "      WHEN LOWER(COALESCE(p.status, '')) = 'w locie' THEN
14     "          'Dostepny' "
15     "      ELSE p.status "
16     "END AS status "
17     "FROM planes p "
18     "ORDER BY p.id DESC"
19 );
20 while (query.next()) {
21     QVariantMap map;
22     map["id"] = query.value("id");
23     map["brand"] = query.value("brand");
24     map["model"] = query.value("model");
25     map["status"] = query.value("status");
26     map["thrust"] = query.value("thrust");
27     map["length"] = query.value("length");
28     map["numberOfEngines"] = query.value("number_of_engines");
29     map["passengers"] = query.value("passengers");
30     map["maxSpeed"] = query.value("max_speed");
31     map["maxAltitude"] = query.value("max_altitude");
32     map["imagePath"] = query.value("image_path");
33     list.append(map);
34 }
35 return list;
36 }

```

Listing 2: Read dla samolotów: `getAllPlanes`

2.1.3 Update

Aktualizacja danych samolotu wykonywana jest przez `updatePlane` (listing 3). Metoda aktualizuje zarówno pola opisowe, jak i parametry techniczne, a ścieżka do grafiki jest zapisywana jako NULL w przypadku pustej wartości.

```

1 bool PlaneManager::updatePlane(int id, const QString &brand, const
   QString &model, const QString &status,
2                                   int thrust, double length, int
                                   numberOfEngines,
3                                   int passengers, double maxSpeed,
                                   double maxAltitude, const QString
                                   &imagePath) {
4     QSqlQuery query;
5     query.prepare("UPDATE planes SET brand = :brand, model = :model,
        status = :status, thrust = :thrust, length = :length, "

```

```

6         "number_of_engines = :numberOfEngines, passengers =
          :passengers, max_speed = :maxSpeed, max_altitude
          = :maxAltitude, "
7         "image_path = :imagePath WHERE id = :id");
8     query.bindValue(":brand", brand);
9     query.bindValue(":model", model);
10    query.bindValue(":status", status);
11    query.bindValue(":thrust", thrust);
12    query.bindValue(":length", length);
13    query.bindValue(":numberOfEngines", numberOfEngines);
14    query.bindValue(":passengers", passengers);
15    query.bindValue(":maxSpeed", maxSpeed);
16    query.bindValue(":maxAltitude", maxAltitude);
17    query.bindValue(":imagePath", imagePath.isEmpty() ? QVariant() :
        imagePath);
18    query.bindValue(":id", id);
19
20    if (!query.exec()) {
21        qDebug() << "Bład SQL przy aktualizacji samolotu:" <<
            query.lastError().text();
22        return false;
23    }
24    qDebug() << "Pomyślnie zaktualizowano samolot ID:" << id;
25    return true;
26 }

```

Listing 3: Update dla samolotów: updatePlane

2.1.4 Delete

Usuwanie samolotu to proste DELETE po identyfikatorze id (listing 4), również wykonane jako zapytanie parametryzowane.

```

1 bool PlaneManager::deletePlane(int id) {
2     QSqlQuery query;
3     query.prepare("DELETE FROM planes WHERE id = :id");
4     query.bindValue(":id", id);
5
6     if (!query.exec()) {
7         qDebug() << "Bład SQL przy usuwaniu samolotu:" <<
            query.lastError().text();
8         return false;
9     }
10    qDebug() << "Pomyślnie usunięto samolot ID:" << id;
11    return true;
12 }

```

Listing 4: Delete dla samolotów: deletePlane

2.2 Lotniska (airports)

Model danych lotnisk jest prosty (kod ICAO, nazwa, współrzędne), dzięki czemu operacje CRUD są krótkie i czytelne. Dodatkowo w `AirportManager` znajduje się funkcja pomocni-

cza do parsowania linku z Google Maps, co ułatwia dodawanie nowych lotnisk bez konieczności ręcznego wpisywania współrzędnych.

2.2.1 Create

Dodanie lotniska realizuje `addAirport` (listing 5). Zapytanie `INSERT` jest parametryzowane, a wartości są wiązane przez `bindValue`. Dzięki temu można łatwo dodać nowe lotnisko z poziomu UI, podając kod ICAO, nazwę i współrzędne (lub link z Google Maps).

```
1 bool AirportManager::addAirport(const QString &icao, const QString
   &name, double lat, double lon) {
2     QSqlQuery query;
3     query.prepare("INSERT INTO airports (icao_code, name, latitude,
        longitude) "
4         "VALUES (:icao, :name, :lat, :lon)");
5     query.bindValue(":icao", icao);
6     query.bindValue(":name", name);
7     query.bindValue(":lat", lat);
8     query.bindValue(":lon", lon);
9
10    if (!query.exec()) {
11        qDebug() << "Bład SQL przy dodawaniu lotniska:" <<
            query.lastError().text();
12        return false;
13    }
14    qDebug() << "Pomyślnie dodano lotnisko:" << icao;
15    return true;
16 }
```

Listing 5: Create dla lotnisk: `addAirport`

2.2.2 Read

Odczyt listy lotnisk w `getAllAirports` (listing 6) pobiera dane posortowane po `icao_code` i zwraca `QVariantList` map gotową do użycia w QML.

```
1 QVariantList AirportManager::getAllAirports() {
2     QVariantList list;
3     QSqlQuery query("SELECT id, icao_code, name, latitude, longitude
        FROM airports ORDER BY icao_code ASC");
4
5     while (query.next()) {
6         QVariantMap map;
7         map["id"] = query.value("id");
8         map["icaoCode"] = query.value("icao_code");
9         map["name"] = query.value("name");
10        map["latitude"] = query.value("latitude");
11        map["longitude"] = query.value("longitude");
12        list.append(map);
13    }
14    return list;
15 }
```

Listing 6: Read dla lotnisk: `getAllAirports` (tabela `airports`)

2.2.3 Update

Edycja lotniska (listing 7) odbywa się poprzez UPDATE po id i pozwala zmienić wszystkie pola modelu.

```
1 bool AirportManager::updateAirport(int id, const QString &icao, const
  QString &name, double lat, double lon) {
2     QSqlQuery query;
3     query.prepare("UPDATE airports SET icao_code = :icao, name =
      :name, "
4         "latitude = :lat, longitude = :lon WHERE id = :id");
5     query.bindValue(":icao", icao);
6     query.bindValue(":name", name);
7     query.bindValue(":lat", lat);
8     query.bindValue(":lon", lon);
9     query.bindValue(":id", id);
10
11     if (!query.exec()) {
12         qDebug() << "Bład SQL przy edycji lotniska:" <<
            query.lastError().text();
13         return false;
14     }
15     return true;
16 }
```

Listing 7: Update dla lotnisk: updateAirport

2.2.4 Delete

Usuwanie lotniska w deleteAirport (listing 8) wykonuje DELETE po id.

```
1 bool AirportManager::deleteAirport(int id) {
2     QSqlQuery query;
3     query.prepare("DELETE FROM airports WHERE id = :id");
4     query.bindValue(":id", id);
5
6     if (!query.exec()) {
7         qDebug() << "Bład SQL przy usuwaniu lotniska:" <<
            query.lastError().text();
8         return false;
9     }
10    return true;
11 }
```

Listing 8: Delete dla lotnisk: deleteAirport

2.3 Loty (flights)

Przy lotach bardzo ważna jest walidacja reguł biznesowych. Przy dodawaniu i edycji aplikacja sprawdza m.in. czy wybrany samolot nie jest w serwisie oraz czy nie dochodzi do kolizji terminów.

2.3.1 Create

Rezerwacja lotu w `addFlight` (listing 9) wykonuje walidację (sprawdza status serwisowy i konflikt terminów), a następnie zapisuje rekord w bazie. Czas jest konwertowany do UTC przed zapisem.

```
1 bool FlightManager::addFlight(int planeId, int depAirportId, int
   arrAirportId,
2                               const QDateTime &startTime, const
                                   QDateTime &endTime) {
3     const QDateTime startUtc = toDbUtc(startTime);
4     const QDateTime endUtc = toDbUtc(endTime);
5
6     if (isPlaneInService(planeId)) {
7         qDebug() << "Nie mozna zarezerwowac lotu: samolot jest w
           serwisie.";
8         return false;
9     }
10
11     if (hasPlaneScheduleConflict(planeId, startUtc, endUtc)) {
12         qDebug() << "Nie mozna zarezerwowac lotu: samolot ma juz
           zajety termin.";
13         return false;
14     }
15
16     QSqlQuery query;
17     query.prepare("INSERT INTO flights (plane_id,
        departure_airport_id, arrival_airport_id, start_time, end_time)
18
19                 "VALUES (:pId, :depId, :arrId, :start, :end)");
20     query.bindValue(":pId", planeId);
21     query.bindValue(":depId", depAirportId);
22     query.bindValue(":arrId", arrAirportId);
23     query.bindValue(":start", startUtc);
24     query.bindValue(":end", endUtc);
25
26     if (!query.exec()) {
27         qDebug() << "Blad SQL przy dodawaniu lotu:" <<
           query.lastError().text();
28         return false;
29     }
30     qDebug() << "Pomyslne zarezerwowano lot.";
31     return true;
}
```

Listing 9: Create dla lotów: `addFlight`

2.3.2 Read

Odczyt lotów (listing 10) korzysta z `JOIN`, aby zwrócić czytelne dane (np. marka/model samolotu oraz kody ICAO i współrzędne lotnisk). Dodatkowo status lotu (zaplanowany/w trakcie/zakończony) jest wyliczany na podstawie czasu.

```

1  QVariantList FlightManager::getAllFlights() {
2      QVariantList list;
3      QString sql = "SELECT f.id, p.brand, p.model, "
4                     "dep.icao_code as dep_icao, arr.icao_code as
5                     "arr_icao, "
6                     "dep.latitude as dep_latitude, dep.longitude as
7                     "dep_longitude, "
8                     "arr.latitude as arr_latitude, arr.longitude as
9                     "arr_longitude, "
10                     "f.start_time, f.end_time, "
11                     "f.plane_id, f.departure_airport_id,
12                     f.arrival_airport_id "
13                     "FROM flights f "
14                     "JOIN planes p ON f.plane_id = p.id "
15                     "JOIN airports dep ON f.departure_airport_id =
16                     dep.id "
17                     "JOIN airports arr ON f.arrival_airport_id = arr.id "
18                     "ORDER BY f.start_time DESC";
19
20     QSqlQuery query(sql);
21     const QDateTime nowUtc = QDateTime::currentDateTimeUtc();
22     while (query.next()) {
23         const QDateTime startUtc =
24             toUtcForCompare(query.value("start_time"));
25         const QDateTime endUtc =
26             toUtcForCompare(query.value("end_time"));
27
28         QString statusText = "Zaplanowany";
29         QString statusColor = "#2E7D32";
30
31         if (startUtc.isValid() && endUtc.isValid()) {
32             if (nowUtc < startUtc) {
33                 statusText = "Zaplanowany";
34                 statusColor = "#2E7D32";
35             } else if (nowUtc <= endUtc) {
36                 statusText = "W trakcie";
37                 statusColor = "#F9A825";
38             } else {
39                 statusText = "Zakończony";
40                 statusColor = "#C62828";
41             }
42         }
43
44         QVariantMap map;
45         map["id"] = query.value("id");
46         map["planeName"] = query.value("brand").toString() + " " +
47             query.value("model").toString();
48         map["depIcao"] = query.value("dep_icao");
49         map["arrIcao"] = query.value("arr_icao");
50         map["depLatitude"] = query.value("dep_latitude");
51         map["depLongitude"] = query.value("dep_longitude");

```

```

45     map["arrLatitude"] = query.value("arr_latitude");
46     map["arrLongitude"] = query.value("arr_longitude");
47     map["startTime"] = toLocalForDisplay(query.value("start_time"))
48         .toString("dd.MM.yyyy HH:mm");
49     map["endTime"] = toLocalForDisplay(query.value("end_time"))
50         .toString("dd.MM.yyyy HH:mm");
51     map["startTimeUtcMs"] =
52         toUtcForCompare(query.value("start_time"))
53         .toMsecsSinceEpoch();
54     map["endTimeUtcMs"] = toUtcForCompare(query.value("end_time"))
55         .toMsecsSinceEpoch();
56     map["status"] = statusText;
57     map["statusColor"] = statusColor;
58
59     map["planeId"] = query.value("plane_id");
60     map["depAirportId"] = query.value("departure_airport_id");
61     map["arrAirportId"] = query.value("arrival_airport_id");
62
63     list.append(map);
64 }

```

Listing 10: Read dla lotów: getAllFlights

2.3.3 Update

Aktualizacja lotu w `updateFlight` (listing 11) ponownie wykonuje walidację, a następnie zapisuje nowe parametry lotu jako UPDATE. Dzięki temu funkcja edycji jest równie bezpieczna i spójna jak dodawanie nowego lotu.

```

1     return list;
2 }
3
4 bool FlightManager::updateFlight(int id, int planeId, int
5     depAirportId, int arrAirportId,
6         const QDateTime &startTime, const
7         QDateTime &endTime) {
8
9     const QDateTime startUtc = toDbUtc(startTime);
10    const QDateTime endUtc = toDbUtc(endTime);
11
12    if (isPlaneInService(planeId)) {
13        qDebug() << "Nie mozna zaktualizowac lotu: samolot jest w
14            serwisie.";
15        return false;
16    }
17
18    if (hasPlaneScheduleConflict(planeId, startUtc, endUtc, id)) {
19        qDebug() << "Nie mozna zaktualizowac lotu: samolot ma juz
20            zajety termin.";
21        return false;
22    }
23
24    QSqlQuery query;

```

```
20 query.prepare("UPDATE flights SET plane_id = :pId,
21     departure_airport_id = :depId, "
22     "arrival_airport_id = :arrId, start_time = :start,
23     end_time = :end "
24     "WHERE id = :id");
25 query.bindValue(":pId", planeId);
26 query.bindValue(":depId", depAirportId);
27 query.bindValue(":arrId", arrAirportId);
28 query.bindValue(":start", startUtc);
29 query.bindValue(":end", endUtc);
30 query.bindValue(":id", id);
31
32 if (!query.exec()) {
33     qDebug() << "Błąd SQL przy edycji lotu:" <<
34         query.lastError().text();
35     return false;
36 }
```

Listing 11: Update dla lotów: updateFlight

2.3.4 Delete

Usuwanie lotu (listing 12) realizuje DELETE po id i zwalnia termin w planie samolotu.

```
1     return true;
2 }
3
4 bool FlightManager::deleteFlight(int id) {
5     QSqlQuery query;
6     query.prepare("DELETE FROM flights WHERE id = :id");
7     query.bindValue(":id", id);
8
9     if (!query.exec()) {
10         qDebug() << "Błąd SQL przy usuwaniu lotu:" <<
11             query.lastError().text();
12         return false;
13     }
```

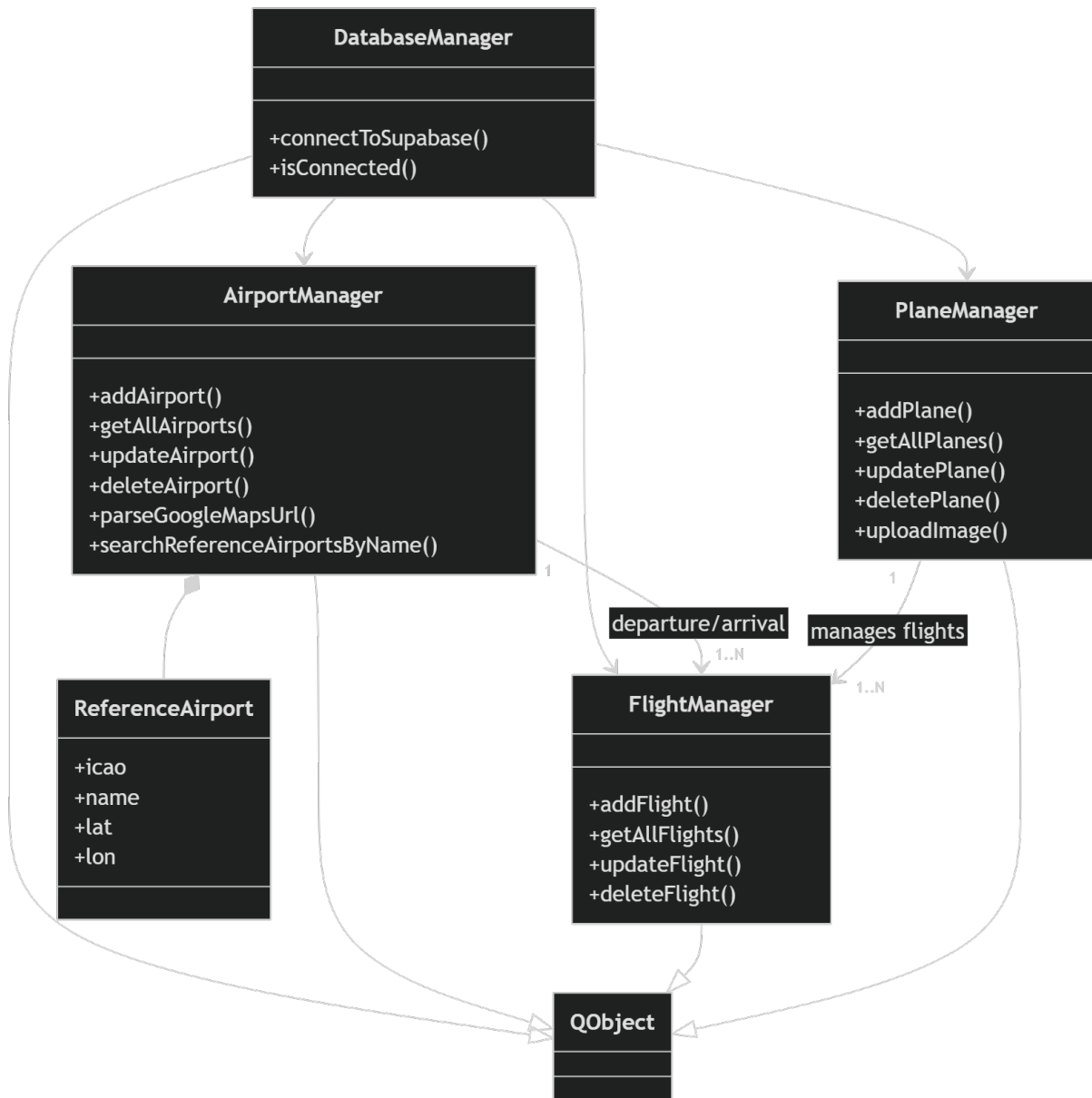
Listing 12: Delete dla lotów: deleteFlight (tabela flights)

3 Struktura aplikacji

Aplikacja została zbudowana w podziale na warstwę logiki w C++ oraz warstwę interfejsu w QML. Ważnym elementem architektury jest sposób komunikacji C++-QML: w tym projekcie klasy nie pełnią roli tradycyjnych obiektów domenowych (z bogatym modelem typów i osobnymi klasami `Plane/Flight/Airport`), tylko są lekkimi serwisami (menedżerami) udostępniającymi zestaw metod do UI.

Diagram klas

Diagram 2 przedstawia główne klasy C++ udostępniane do QML (serwisy) oraz ich relacje. Warto zaznaczyć, że pola typu `id`, `icao_code` itp. nie są atrybutami klas C++. Są to odpowiednio kolumny w bazie danych oraz klucze w mapach zwracanych do QML (`QVariantMap`).



Rysunek 2: Diagram klas aplikacji (serwisy C++ i zależności)

Warstwa C++: serwisy i komunikacja z bazą danych

W części C++ znajdują się klasy dziedziczące po `QObject`, których głównym zadaniem jest udostępnienie API do QML oraz komunikacja z bazą danych:

- **DatabaseManager**: inicjalizuje połączenie z PostgreSQL (Supabase).
- **PlaneManager**: operacje CRUD na samolotach oraz zapytania dla uzyskania danych statystycznych.
- **AirportManager**: operacje CRUD na lotniskach oraz funkcje pomocnicze (np. parsowanie linku z Google Maps i podpowiedzi lotnisk).
- **FlightManager**: operacje CRUD na lotach wraz z walidacją terminów oraz pobieraniem danych z tabel samolotów i lotnisk.

Klasy pełnią rolę warstwy pośredniej pomiędzy UI a bazą danych: wywołanie metody w QML uruchamia zapytanie SQL, a wynik jest konwertowany do typu zrozumiałego dla QML.

Udostępnienie serwisów do QML

W projekcie nie tworzą własnych modeli QML (np. `QAbstractListModel`). Zamiast tego serwisy z C++ są dodane do kontekstu jako `contextProperty`. Dzięki temu QML widzi je jako dostępne globalnie obiekty (np. `planeService`) i może wywoływać ich metody `Q_INVOKABLE`.

```

1 DatabaseManager dbManager;
2 dbManager.connectToSupabase(); // Test polaczenia z baza danych
3 engine.rootContext()->setContextProperty("myDb", &dbManager);
4
5 // Udostepniamy obiekt dla QML
6 PlaneManager planeManager;
7 engine.rootContext()->setContextProperty("planeService",
8     &planeManager);
9
10 AirportManager airportManager;
11 engine.rootContext()->setContextProperty("airportService",
12     &airportManager);
13
14 FlightManager flightManager;
15 engine.rootContext()->setContextProperty("flightService",
16     &flightManager);
17
18 const QUrl url(u"qrc:/qt/qml/PlaneManager/qml/main.qml"_s);
19 QObject::connect(&engine, &QQmlApplicationEngine::objectCreated,
20     &app, [url](QObject *obj, const QUrl &objUrl) {
21         if (!obj && url == objUrl) QCoreApplication::exit(-1);
22     }, Qt::QueuedConnection);
23
24 engine.load(url);
25
26 return app.exec();
27 }
```

Listing 13: Udostępnienie obiektów serwisów w kontekście QML (fragment `main.cpp`)

Wymiana danych przez `QVariantList` i `QVariantMap`

Kluczową cechą tej architektury jest dobór typów zwracanych przez metody serwisów. QML operuje na dynamicznych strukturach podobnych do JavaScript (tablice i obiekty), dlatego w C++ zwracane są typy generyczne:

- `QVariantList`: lista rekordów (np. lista samolotów lub lotów),
- `QVariantMap`: pojedynczy rekord lub obiekt ze zbiorem pól (klucze–wartości),

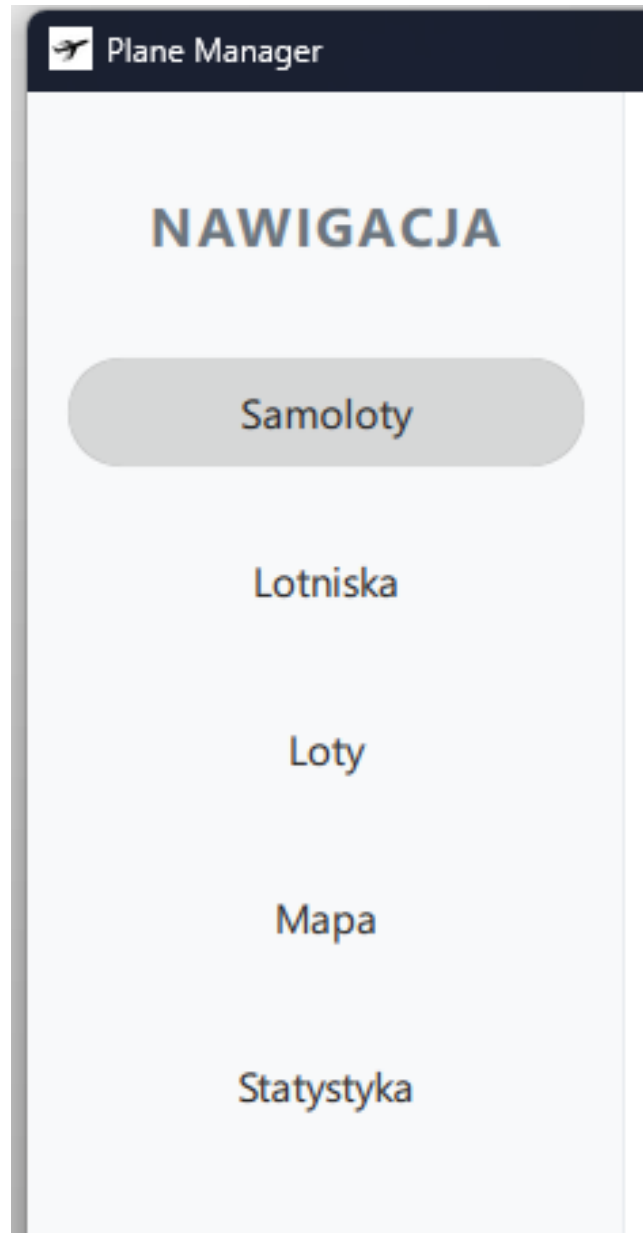
W praktyce wygląda to następująco: C++ wykonuje zapytanie `QSqlQuery`, dla każdego wiersza tworzy mapę (np. `{id, brand, model, status}`), a następnie dodaje ją do listy. Po stronie QML rezultat jest przypisywany bezpośrednio jako model widoku listy, a w delegacie elementy są odczytywane przez klucze mapy (np. `modelData.brand`, `modelData.status`). To podejście znacząco upraszcza integrację z UI.

4 Działanie aplikacji

Poniżej przedstawiono najważniejsze widoki aplikacji wraz z krótkim opisem ich działania. Każdy zrzut ekranu pokazuje osobną funkcję (lub grupę funkcji) tak, aby w raporcie było widać pełny zakres możliwości aplikacji.

4.1 Ekran główny: nawigacja pomiędzy modułami

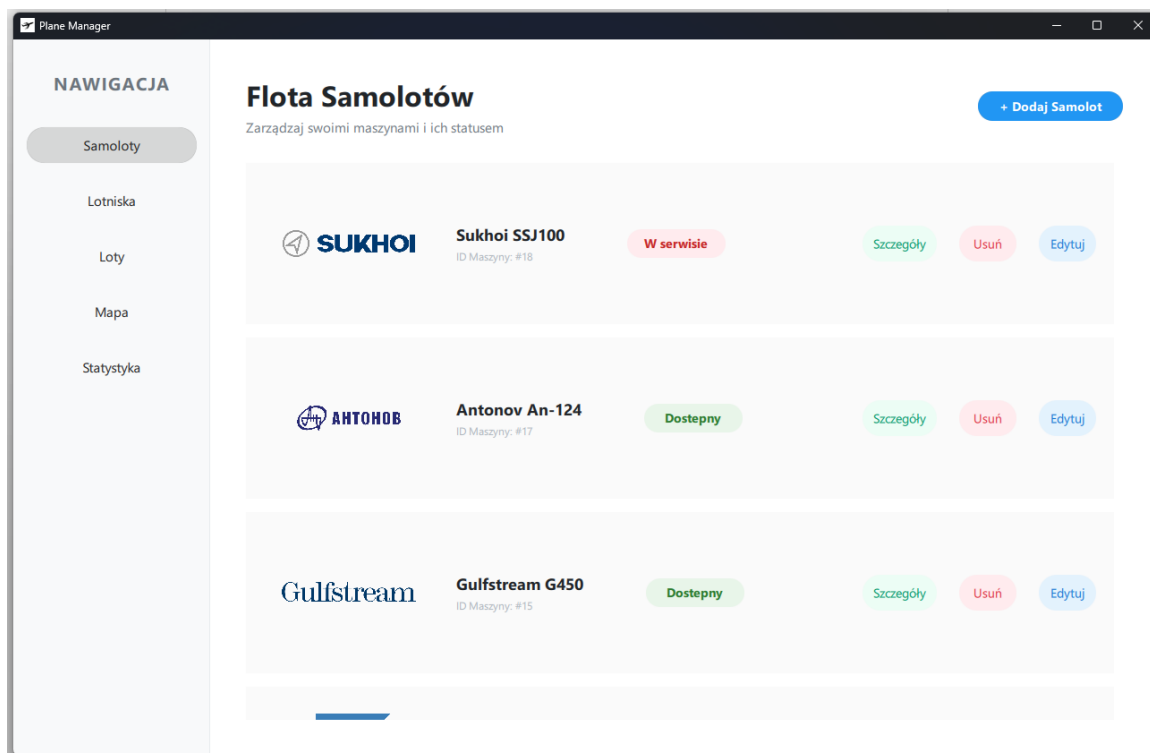
Na ekranie głównym widoczny jest panel boczny z nawigacją oraz obszar roboczy. Aplikacja udostępnia pięć głównych modułów: *Samoloty*, *Lotniska*, *Loty*, *Mapa* oraz *Statystyka*. Przełączanie widoków realizowane jest przez `StackView` w QML.



Rysunek 3: Ekran główny aplikacji z panelem nawigacji

4.2 Samoloty: lista floty (odczyt danych)

Widok listy samolotów prezentuje bieżącą flotę wraz ze statusem każdej maszyny. Dane są pobierane z serwisu C++ i przekazywane do QML jako lista map (`QVariantList` / `QVariantMap`), dzięki czemu mogą zostać użyte bezpośrednio jako model `ListView`.



Rysunek 4: Widok listy samolotów (Read)

4.3 Samoloty: dodawanie nowej maszyny (Create)

Przycisk *+ Dodaj Samolot* otwiera formularz, w którym można wprowadzić podstawowe dane (marka, model, status) oraz parametry techniczne. Po zatwierdzeniu formularza wywoływana jest metoda serwisu C++ wykonująca operację `INSERT` w bazie danych.

The screenshot shows the 'Dodaj Nowy Samolot' (Add New Plane) dialog box in the Plane Manager application. The dialog is overlaid on a blurred background of the main application interface, which includes a sidebar with navigation links (Samoloty, Lotniska, Loty, Mapa, Statystyka) and a main area displaying a list of planes with logos for Sukhoi, Antonov, and Gulfstream. The dialog itself has a title bar and contains two sections: 'Podstawowe dane' (Basic data) and 'Parametry techniczne' (Technical parameters). The 'Podstawowe dane' section includes input fields for 'Marka (np. Airbus)', 'Model (np. A320)', and a dropdown menu for 'Dostępny'. The 'Parametry techniczne' section includes input fields for 'Siła ciągu (kN)', 'Długość (m)', 'Liczba silników', 'Pasażerowie', 'Max prędkość (km/h)', and 'Max wysokość (km)'. The dialog has 'Anuluj' (Cancel) and 'OK' buttons at the bottom right.

Rysunek 5: Dialog dodawania samolotu (Create)

4.4 Samoloty: edycja i usuwanie (Update/Delete)

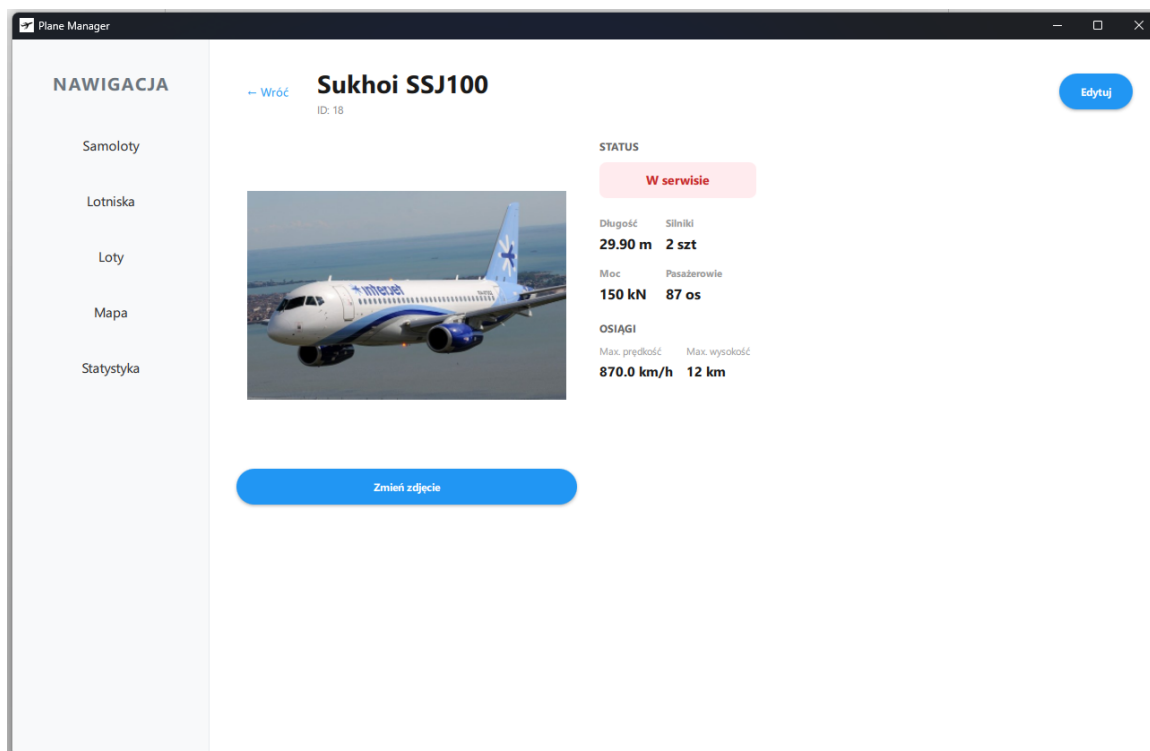
Z poziomu listy można otworzyć dialog edycji (modyfikacja danych w bazie przez UPDATE) oraz usunąć rekord (DELETE). Dzięki temu moduł samolotów realizuje pełne CRUD na tabeli planes.

The screenshot shows the 'Edycja Samolotu' (Edit Plane) dialog box in the Plane Manager application. The dialog is overlaid on a blurred background of the main application interface, which includes a sidebar with navigation links (Samoloty, Lotniska, Loty, Mapa, Statystyka) and a main area displaying a list of planes with logos for Sukhoi, Antonov, and Gulfstream. The dialog itself has a title bar and contains two sections: 'Podstawowe dane' (Basic data) and 'Parametry techniczne' (Technical parameters). The 'Podstawowe dane' section includes input fields for 'Marka (np. Airbus)' with the value 'Sukhoi', 'Model (np. A320)' with the value 'SSJ100', and a dropdown menu for 'Dostępny' with the value 'W serwisie'. The 'Parametry techniczne' section includes input fields for 'Siła ciągu (kN)' with the value '150', 'Długość (m)' with the value '29.90', 'Liczba silników' with the value '2', 'Pasażerowie' with the value '87', 'Max prędkość (km/h)' with the value '870.00', and 'Max wysokość (km)' with the value '12.20'. The dialog has 'Anuluj' (Cancel) and 'OK' buttons at the bottom right.

Rysunek 6: Dialog edycji samolotu (Update) oraz przyciski usuwania w widoku listy (Delete)

4.5 Samoloty: szczegóły i zarządzanie zdjęciem

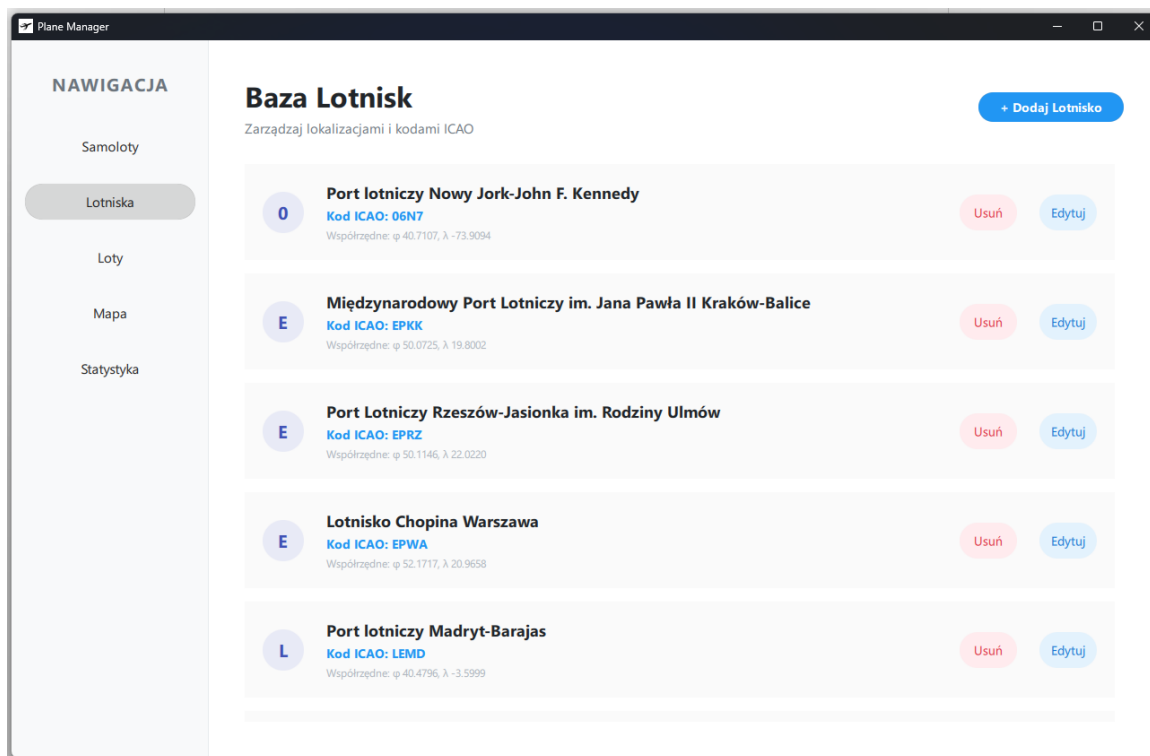
Po wybraniu opcji *Szczegóły* wyświetlany jest ekran z informacjami o konkretnym samolocie. Użytkownik może zmienić zdjęcie maszyny poprzez wybór pliku graficznego z dysku i przesłanie go do aplikacji, a następnie obraz jest wyświetlany w widoku szczegółów.



Rysunek 7: Widok szczegółów samolotu wraz z obsługą zdjęcia

4.6 Lotniska: lista i zarządzanie rekordami (Read/Update/Delete)

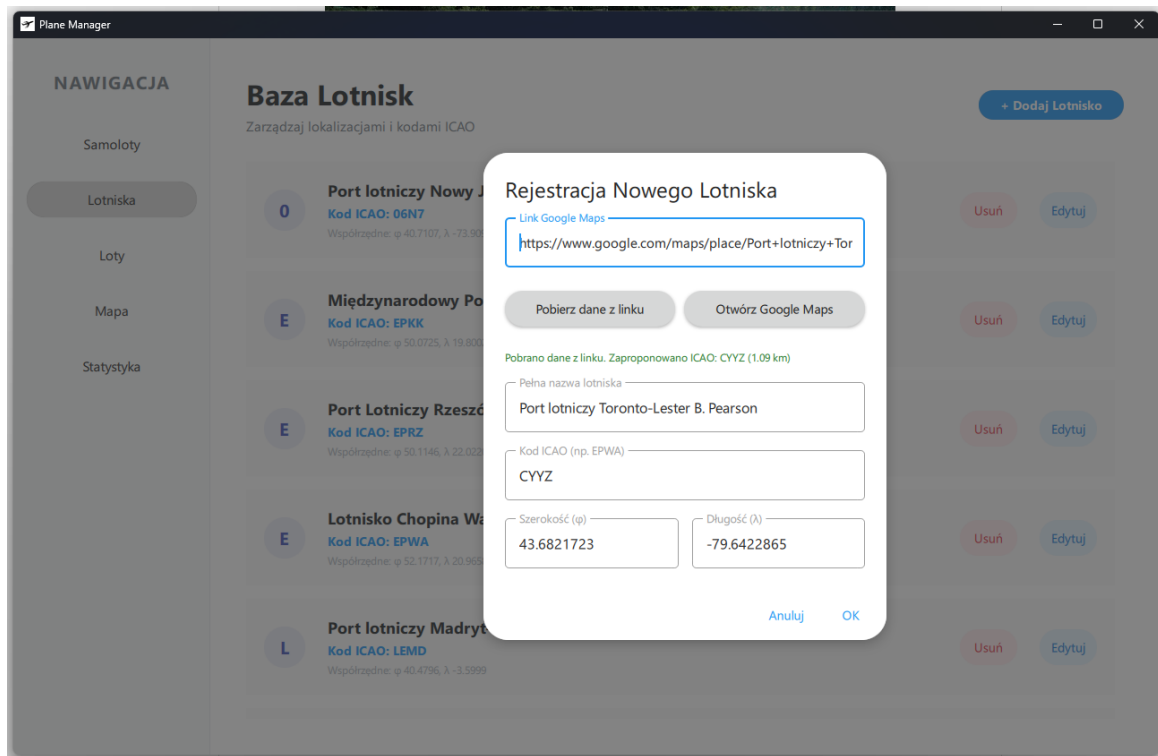
Moduł lotnisk pozwala przeglądać bazę lotnisk oraz edytować i usuwać wpisy. Każdy rekord zawiera kod ICAO, nazwę oraz współrzędne geograficzne, które są wykorzystywane m.in. w module mapy.



Rysunek 8: Widok listy lotnisk z opcjami edycji i usuwania

4.7 Lotniska: dodawanie z pomocą linku Google Maps i podpowiedzi

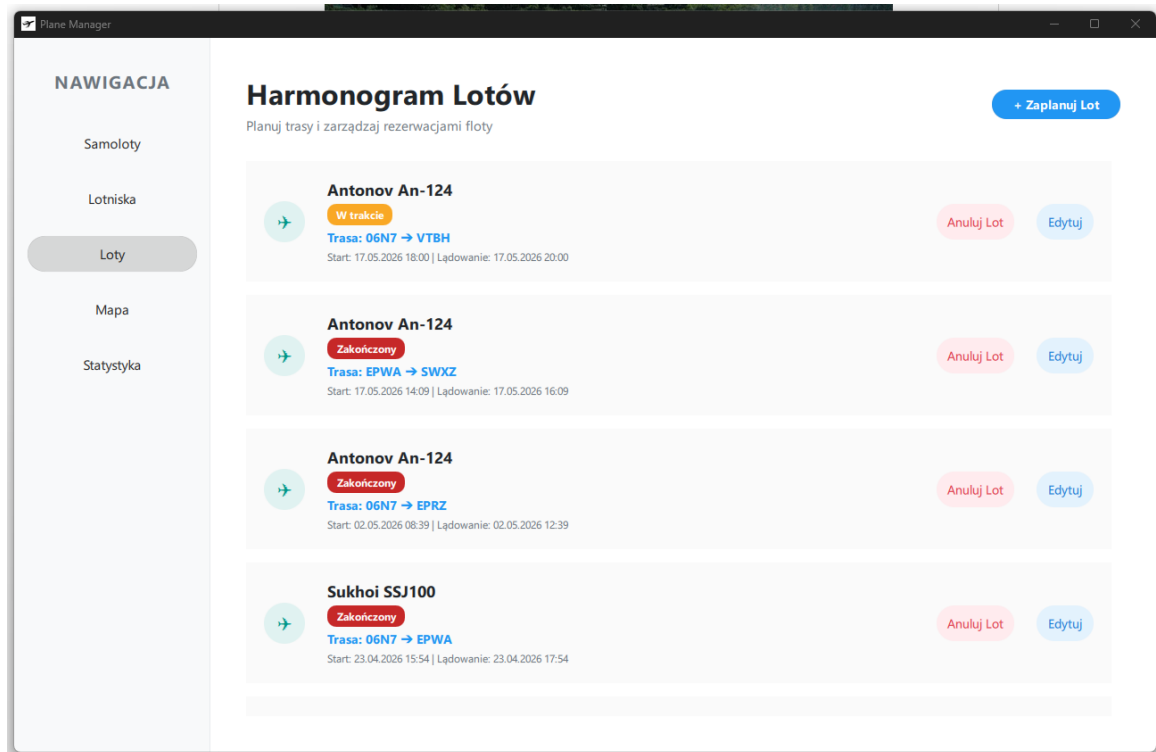
Podczas dodawania lotniska dostępne jest wsparcie wprowadzania danych: aplikacja potrafi odczytać nazwę i współrzędne z linku Google Maps, a także wyświetla podpowiedzi lotnisk po wpisaniu fragmentu nazwy. Funkcje te ograniczają liczbę błędów i przyspieszają tworzenie nowych rekordów.



Rysunek 9: Dialog dodawania lotniska: pobranie danych z linku i lista podpowiedzi

4.8 Loty: harmonogram (Read) i anulowanie rezerwacji (Delete)

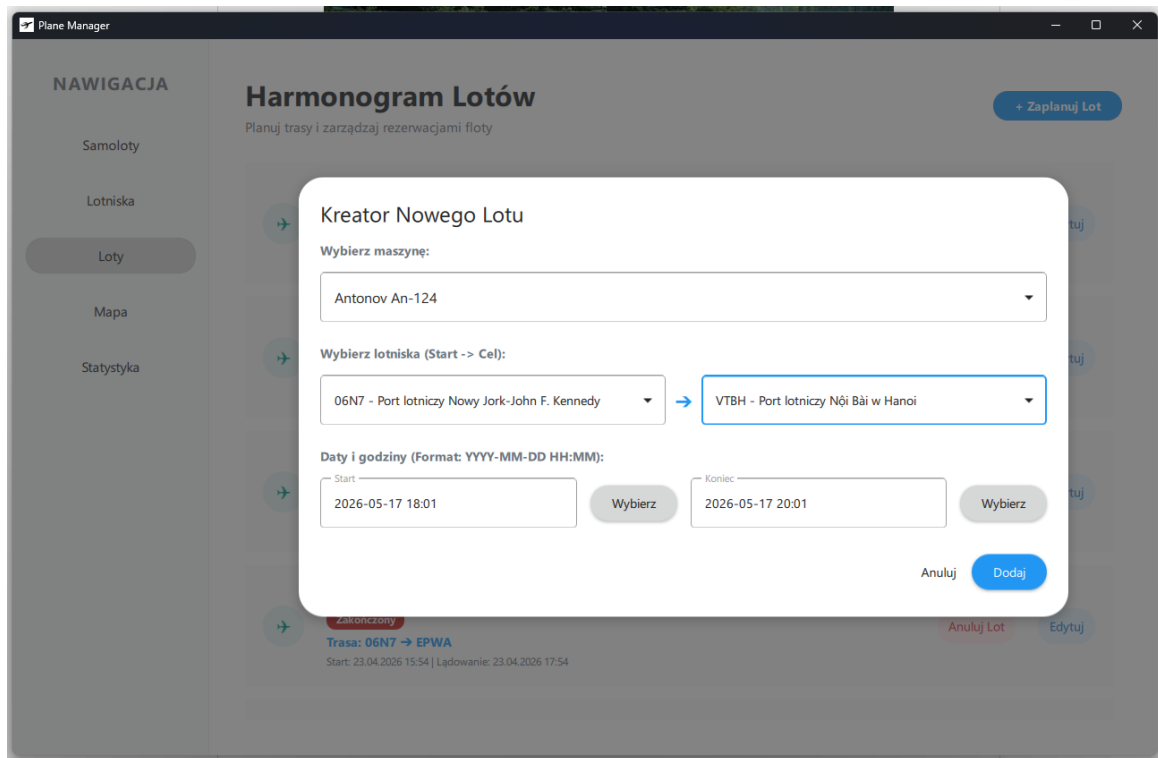
Widok lotów prezentuje harmonogram rezerwacji floty wraz z trasą (kodami ICAO) oraz godzinami startu i lądowania. Z poziomej listy można również anulować lot, co usuwa rekord z bazy i zwalnia termin samolotu.



Rysunek 10: Widok listy lotów (Read) oraz przycisk anulowania (Delete)

4.9 Loty: planowanie i edycja lotu (Create/Update) oraz wybór daty

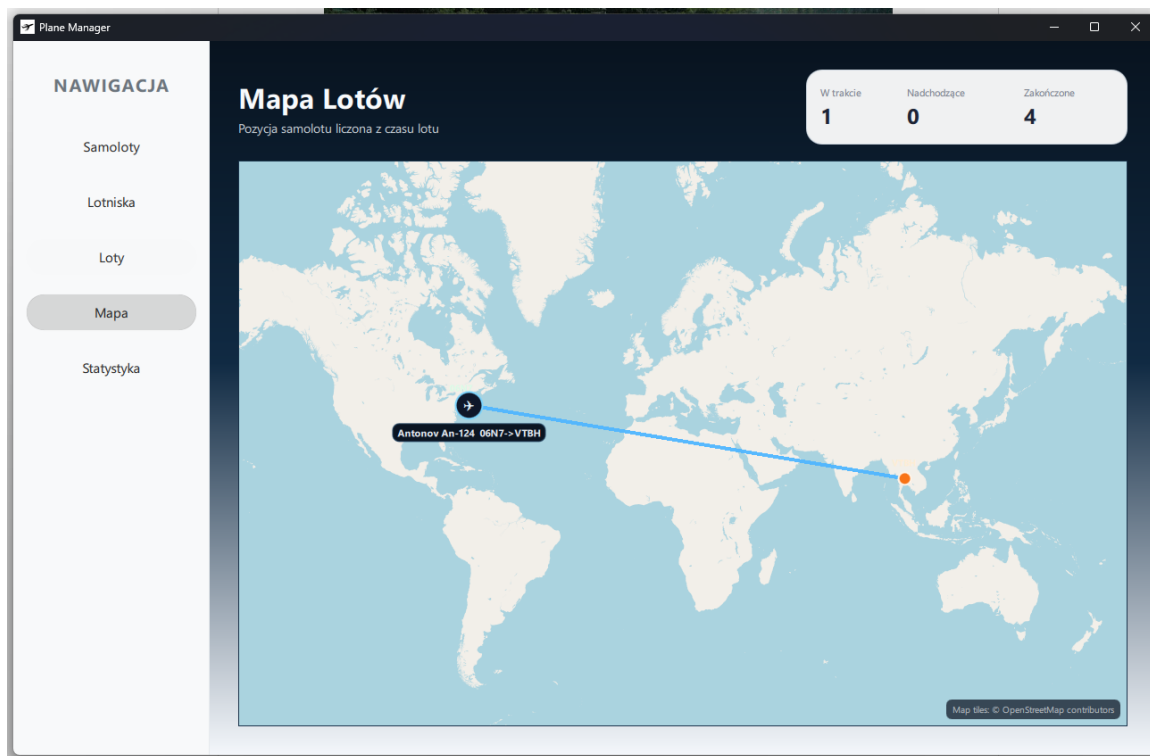
Przy planowaniu i edycji lotu użytkownik wybiera samolot oraz lotniska wylotu i przylotu z list rozwijanych, a daty ustawia przez wbudowany selektor daty i czasu. Po zatwierdzeniu wykonywana jest walidacja w warstwie C++ (np. konflikt terminów lub samolot w serwisie), a następnie zapis do bazy.



Rysunek 11: Dialog planowania/edycji lotu z otwartym selektorem daty i czasu

4.10 Mapa lotów

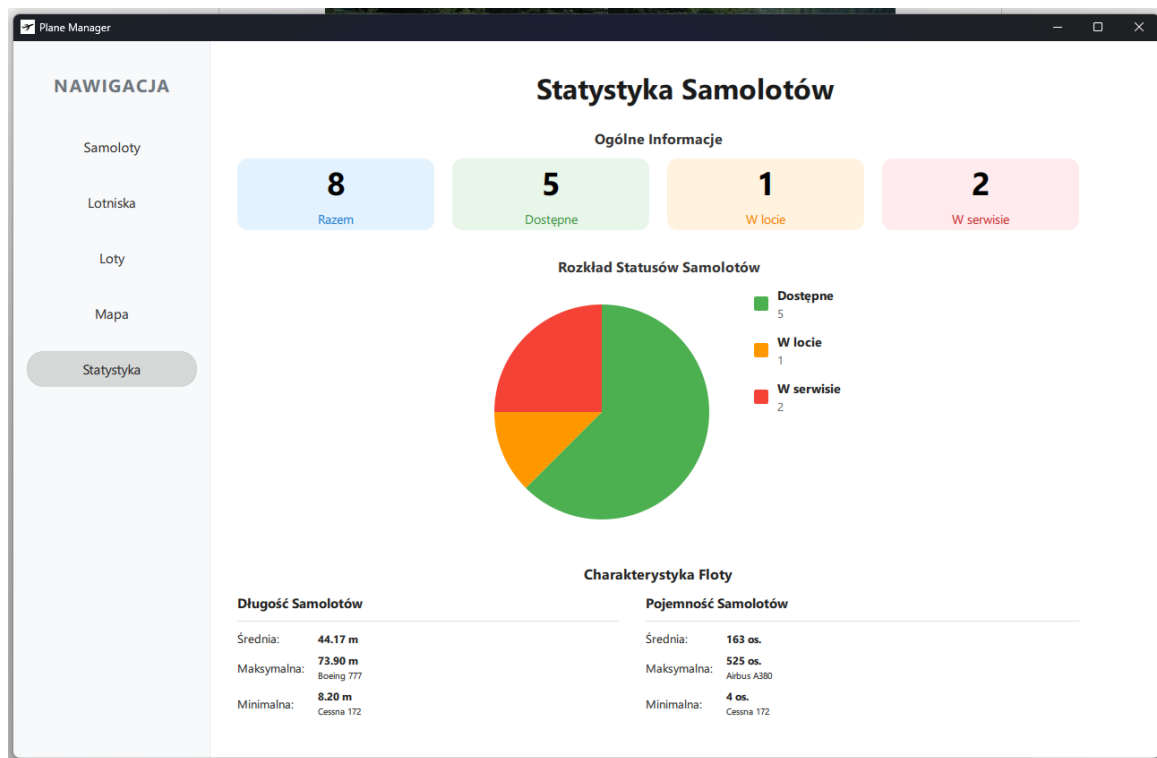
Widok mapy prezentuje aktualnie trwające loty na podkładzie OpenStreetMap. Pozycja samolotu jest wyliczana na podstawie czasu (procent przebiegu pomiędzy startem a końcem), a trasa jest rysowana jako linia pomiędzy lotniskami. Dodatkowo widoczne są liczniki lotów w trakcie, nadchodzących i zakończonych.



Rysunek 12: Mapa lotów z trasą, markerami oraz licznikami

4.11 Statystyka floty

Moduł statystyk prezentuje podsumowanie floty: liczbę samolotów w poszczególnych statusach oraz wykres kołowy rozkładu. Dodatkowo wyświetlane są wartości ekstremalne (np. najdłuższy i najkrótszy samolot), co ułatwia szybki przegląd zasobów.



Rysunek 13: Widok statystyk floty (podsumowania i wykres)

5 Podsumowanie

W tym miejscu należy podsumować projekt. Warto opisać co było najtrudniejszym elementem, jaka część była najciekawsza, a która pozwoliła się najwięcej nauczyć.

Jeżeli na jakimkolwiek etapie wykorzystywana była sztuczna inteligencja generatywna, to w tym miejscu należy również zamieścić link udostępniający pełną przeprowadzoną rozmowę(y). Link należy wygenerować, na samym końcu pisania raportu. Link powinien działać w momencie oceny pracy oraz w trakcie obrony projektu.